

VINUNIVERSITY

Vector Store & Feature Store

AICB-P2T2 · Ngày 19 · Chương 4: Hạ Tầng

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 4

HÃY SUY NGHĨ...

“SQL database trả về exact match. Nhưng AI cần “tương tự” – semantic search. Tại sao SQL không đủ cho AI search, và vector database thực sự lưu gì?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Vector Embeddings: Text → Số
2. Vector DB Landscape (3 tầng lưu trữ)
3. ANN, Filtered Search & Hybrid
4. RAG Pipeline & Long Context
5. GraphRAG & Knowledge Graphs
6. Agentic Retrieval & Agent Memory (code)
7. Feature Engineering & Feature Store (code)
8. Production: bảo mật, chi phí, case studies
9. Ứng dụng: Wiki / CodeWiki / DocWiki
10. Demo: Semantic Search API

Mục tiêu bài học

Sau buổi học này, bạn sẽ:

1. Hiểu vector embeddings và chọn model **bằng bằng chứng** (VN-MTEB)
2. Deploy và query vector database — và chọn đúng **tầng lưu trữ** (RAM / SSD / object storage)
3. Xây dựng RAG pipeline với hybrid search, filtered search và reranking
4. Setup Feast Feature Store với offline/online store
5. Biết rủi ro **bảo mật & tuân thủ** của một kho vector (OWASP LLM09:2026, PDPL)

Agenda hôm nay

Embeddings → Vector DB → ANN + Hybrid → RAG → GraphRAG → Agentic & Memory → Feature Store → Production

Deliverable Cuối Ngày

VINUNIVERSITY

Artifact cần nộp

Vector search endpoint + Feature Store offline/online + 4 mission nâng cao

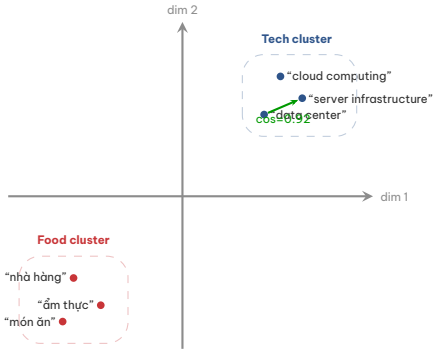
Core (NB1–NB4)

- REST `/search?q=...` top-10, P99 $<$50 ms
- Hybrid: vector + BM25, merge bằng RRF
- Feast: 3 feature views, materialize, online lookup
- Benchmark keyword vs semantic vs hybrid

Nâng cao (NB5–NB8)

- Đo recall cliff của filtered search
- Agent: retrieval-as-tool + tách câu hỏi
- Semantic cache: sweep ngưỡng + demo rò tenant
- Feature engineering: leakage + on-demand view

Text → Dense Vector: Embedding Space

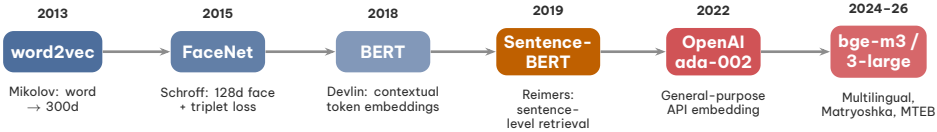


Embedding Models

- `text-embedding-3-small`: OpenAI, 1536d
- `text-embedding-3-large`: 3072d, Matryoshka
- `bge-m3`: BAAI, 1024d, multilingual
- `nomic-embed`: open-source
- PhoBERT: VN lightweight baseline (xem bảng 2026)

Cosine Similarity — $\frac{\vec{A}\cdot\vec{B}}{|\vec{A}|\times|\vec{B}|}$ — giá trị $[-1, 1]$ (thực tế với text embedding thường > 0); ngưỡng “giống” tuỳ model

Embeddings: A Decade of Representations (2013 → 2026)



Key shift over a decade

2013–2019: task-specific embeddings — mỗi domain (NLP, face, audio) train riêng. **2020–2026:** general-purpose foundation embeddings — một model, nhiều use cases, multilingual, multi-modal. Re-index cost cao ⇒ chọn cẩn thận từ đầu.

Embedding Models 2026: Lựa Chọn Theo Use Case

Model (2026)	Dim	Giá	Chọn khi
`gemini-embedding-2`	3072	API	đa phương thức
`text-embedding-3-large`	3072	$0.13/M	chất lượng cao
`text-embedding-3-small`	1536	$0.02/M	baseline rẻ
`cohere embed-5`	$\leq$2048	API	text+ảnh, 128K
`bge-m3`	1024	self-host	**tiếng Việt**
`Qwen3-Embedding`	1024+	self-host	OSS đa ngữ

Tiếng Việt: xem slide VN-MTEB tiếp theo. **Bảng này sẽ cũ trong vài tuần** — học cách chọn, đừng học thuộc.

Matryoshka: Giảm Index 12×

- `text-embedding-3-large` dùng MRL — truncatable: 3072d → 256d
- **256d vẫn thắng ada-002 @ 1536d** trên MTEB
- 3072→256 = RAM/disk nhỏ **12**× (so với ada-002 1536d: 6×)
- Áp dụng ngay khi scale >10M vectors

Lưu ý: **Vệ sinh benchmark:** MTEB v2 (Eng) và MMTEB (đa ngữ) là *hai bảng khác nhau*; **điểm v2 không so được với v1**. Luôn re-rank trên **golden set của bạn** (Ngày 14). Đổi model = re-index toàn bộ.

Tiếng Việt: Chọn Embedding Bằng Bằng Chứng (VN-MTEB)

Model	Params	Dim	VN-MTEB
`bge-m3`	568M	1024	**64.90**
`Vietnamese_Embedding`	568M	1024	63.34
`halong_embedding`	278M	768	61.60
`vietnamese-bi-encoder`	135M	768	54.89

VN-MTEB (arXiv 2507.21500, 07/2025): 41 datasets, 6 tasks, 18 models.

Ba điều bảng này dạy ta

1. Generalist đa ngữ (`bge-m3`) **thắng cả bản fine-tune tiếng Việt** của chính nó trên benchmark rộng
2. Nhưng fine-tune vẫn có thể thắng trên *domain hẹp* ⇒ phải đo trên dữ liệu của bạn
3. Model dùng **RoPE** vượt model dùng absolute positional embedding — lý do kỹ thuật, không phải "mới hơn"

Lưu ý: VN-MTEB được **dịch** từ MTEB tiếng Anh ⇒ đo năng lực *ngôn ngữ*, chưa đo *văn hoá/domain* Việt Nam. Reranker tiếng Việt: `ViRanker` (BGE-M3). Kết luận không đổi: **golden set của bạn là trọng tài cuối cùng**.

Similarity Metrics: Cosine vs Dot vs Euclidean

VINUNIVERSITY

Cosine Similarity

$$\cos(\vec{a}, \vec{b}) = (\vec{a} \cdot \vec{b}) / (\|\vec{a}\| \, \|\vec{b}\|)$$

- Range $[-1, 1]$, angle only
- Default cho text embedding
- OpenAI/BGE/e5 unit-norm $\Rightarrow$ dot = cosine
- Match metric với pretraining objective: sai metric degrade recall 10–20%

Dot Product

$$\vec{a} \cdot \vec{b} = \sum a_i b_i$$

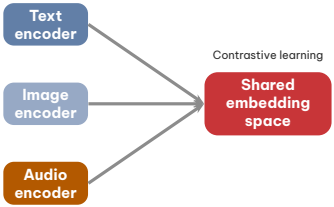
- Bao gồm magnitude
- Nhanh hơn (no division)
- ColBERT, DPR dùng dot

Euclidean (L_2)

$$d = \sqrt{\sum (a_i - b_i)^2}$$

- Triangle inequality $\Rightarrow$ IVF / kd-tree
- CLIP image, wav2vec audio
- Unit-norm: $d^2 = 2(1 - \cos)$

Multi-modal Embeddings: Vector Vượt Khỏi Text



Models 2026

CLIP (OpenAI): text+image, foundational

SigLIP (Google): backbone của ColPali

jina-clip-v2: multilingual incl. tiếng Việt

ImageBind (Meta): 6 modalities

Gemini Embedding 2 (GA 04/2026): text + ảnh + **video** + **audio** + PDF vào *một* không gian 3072d, 100+ ngôn ngữ, MRL truncate. 1 request: 8K tokens / 6 ảnh / 120 s video / 180 s audio / 6 trang PDF.

Use cases 2026

Visual search (Tiki, Shopee) · **Content moderation** (ảnh+caption) · **Video indexing** (tìm clip theo prompt) · **Medical** (X-ray+report đồng embedding).

Vector DB So Sánh

Feature	Pinecone	Weaviate	Qdrant	pgvector
Hosting	Managed	Self/Cloud	Self/Cloud	Extension
API	REST/gRPC	GraphQL	REST/gRPC	SQL
Multi-modal	×	✓	×	×
Filtering	Metadata	GraphQL	Payload	SQL WHERE
P99 (1M vecs)	8–22 ms	~10 ms	~5 ms	15 ms
Quantization	✓	✓	✓ (binary)	scalar
Best for	Production	Multi-modal	Self-hosted	Have Postgres

Nguồn:

VectorDBBench 2025 · 1M vectors, 768 dims, HNSW index, top-10 query. pgvector+pgvectorscale (Timescale DiskANN).

FAISS (Meta, 2017)

- Library, *not* a database — C++/Python, no server
- GPU: 1B vectors, top-K <1s
- IVF, HNSW, PQ, OPQ, IVF-PQ — cùng một API
- In-memory + on-disk indices
- 2026: vẫn là thư viện index bên trong Milvus (và nhiều hệ khác)

Pre-LLM Use Cases

- **Face recognition:** FaceNet, ArcFace — FaceID, photo search
- **Pinterest visual lens:** CNN feats + FAISS, 2017+
- **RecSys candidate gen:** YouTube/Spotify/TikTok — ~1B vectors
- **Plagiarism / dedup:** Turnitin, news article clustering
- **Audio fingerprinting:** Shazam, Spotify duplicate detection

Deep Metric Learning — Pre-LLM vector retrieval dựa vào triplet loss (FaceNet), contrastive loss, ArcFace angular margin — train embedding sao cho *cosine* (hoặc Euclidean) tách class. Đây là tiền thân của contrastive pre-training hiện đại (CLIP, SigLIP).

Ba Tầng Lưu Trữ Vector: RAM / SSD / Object Storage

Tầng	Vector nằm ở	Latency	Chi phí	Dùng khi
In-memory (HNSW)	RAM	5–20 ms	cao nhất	hot, latency-critical, $\leq$10M
On-disk (DiskANN)	SSD cục bộ	$\sim$30 ms	trung bình	100M+, vẫn cần tương tác
Object storage	**S3 / GCS**	$\sim$**100 ms**	**thấp nhất**	corpus khổng lồ, query thưa

Amazon S3 Vectors (GA 12/2025)

- Object storage đầu tiên *native* lưu + query vector
- **2 tỷ vector/index**, 10.000 index/bucket
- Chi phí upload+lưu+query **giảm tới 90%**
- Query thưa $<$1 s; query thường xuyên $\sim$100 ms
- Tích hợp Bedrock KB, OpenSearch, SageMaker

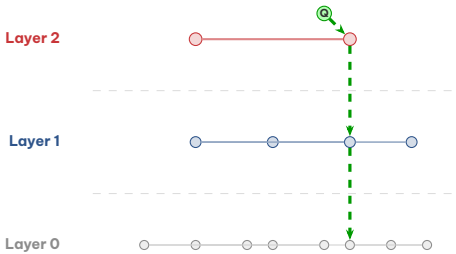
Vì sao rẻ hơn hẳn?

Không phải thuật toán — mà là **giá lưu trữ**: object storage $\sim$\$0,02/GB so với RAM $\sim$\$2+/GB $\Rightarrow$ chênh **hai bậc độ lớn**.

Cùng tầng: **turbopuffer**, **LanceDB** (scale-to-zero khi rảnh).

Đánh đổi: latency cao hơn, QPS thấp hơn.

HNSW: Hierarchical Navigable Small World



HNSW: Vì sao là default 2026

- Graph-based, multi-layer skip list
- Recall 95%+ ở ∼10 ms (in-memory)
- Params: `ef=200, m=16` (default tốt)
- Có sẵn: Weaviate, Qdrant, Milvus, pgvector

Khi HNSW không phải lựa chọn

- RAM-bound: 10M vec @ 768d fp32 → ∼31 GB + graph
- Update-heavy corpus: re-build chậm
- Static, billion-scale → DiskANN (slide tiếp)

HNSW vs IVF vs DiskANN: Khi Nào Dùng Gì

VINUNIVERSITY

HNSW

- Graph in-memory, multi-layer skip list
- Recall 95%+ @ ~10 ms
- RAM-bound: 3 KB/vec (768d × 4 B)
- **Best for:** ≤10M hot vectors, latency-critical
- Params: `ef=200, m=16`

IVF (FAISS)

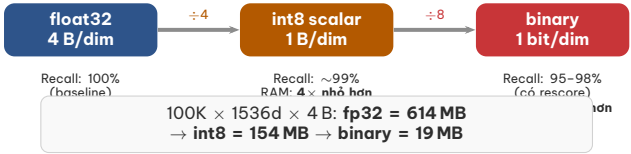
- Cluster-based inverted index
- Lower RAM, higher latency (~30–50 ms)
- Retraining cần khi data drift lớn
- **Best for:** batch-mode, static corpus, CPU-only infra
- Params: `nlist=1024, nprobe=64`

DiskANN

- Graph trên SSD — billion-scale 1 node
- Latency ~30 ms @ 99% recall
- **10–50× rẻ hơn HNSW** ở scale lớn
- Dùng bởi: pgvectorscale (StreamingDiskANN), Milvus, Azure Cosmos DB
- **Best for:** >100M vectors, cost-sensitive

RaBitQ (SIGMOD 2024) — Xoay ngẫu nhiên rồi 1 bit/chiều, có **cận sai số** — VectorChord, Milvus; BBQ (Elasticsearch) phát triển từ đó. **pgvectorscale dùng SBQ**, không phải RaBitQ.

Quantization: Bí Quyết Tiết Kiệm Bộ Nhớ 32×



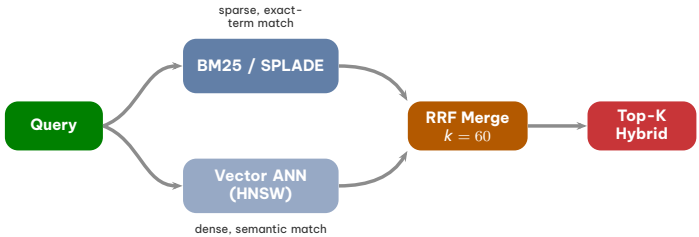
Asymmetric + rescoring (Qdrant 1.19)

- **Lưu binary** (19 MB) + **query giữ độ chính xác cao**
- Best of both: cheap storage + high recall
- Oversample + rescore bằng vector gốc ⇒ recall gần scalar. 2026: TurboQuant (1.18), 4-bit (1.19)

Production Default 2026

- Qdrant, Weaviate, Pinecone, Milvus: hỗ trợ binary/scalar out-of-the-box
- Bật quantization **trước khi ingest** — không phải sau
- Re-quantize = re-index ⇒ blue-green (Production Patterns)

Hybrid Search: BM25 + Vector + RRF



Reciprocal Rank Fusion —

$$\text{score}(d) = \sum_r \frac{1}{k + \text{rank}_r(d)}$$ ($k = 60$; Qdrant mặc định $k = 2$)

Rank-only: không cần normalize raw scores giữa BM25 (TF-IDF) và cosine.

Production 2026 (Hybrid wins)

- Hơn dense-only bao nhiêu? **Đo trên golden set của bạn** — Lab 19: hybrid 78,6% vs semantic 73,2% (topic P@10)
- Lab 19 đo P99: hybrid 8,9 ms vs semantic 6,3 ms (CPU, 1K doc)
- Native: Qdrant, Weaviate, OpenSearch, Milvus
- **SPLADE (Anh)**: thua BM25 trên **10/10** bộ tiếng Việt (slide sau)

Hybrid Cho Tiếng Việt: Bằng Chứng Từ 10 Bộ Dữ Liệu

nDCG@10 (%) — Nguyen & Quan, *EACL 2026 Findings* (HCMUT, VinUni CAIR)

Phương pháp	EduCoQA	ViRHE4QA	CSConDa
BM25	27,9	**80,3**	26,3
SPLADE-v3 (tiếng Anh)	19,9	52,8	14,9
Dense `3-large`	35,5	70,7	44,8
Dense + BM25 (α-fusion)	**38,4**	**81,3**	**47,6**

Fusion (dense = `Vietnamese_Embedding_v2`):

CSConDa: α-fusion **45,2** vs RRF 41,0 ViRHE4QA: **85,2** vs 83,0

Bốn bài học

- SPLADE-v3 thua BM25 trên **10/10** bộ: learned sparse phải *đúng ngôn ngữ*
- BM25 thắng `3-large` ở **4/10** bộ (luật, tin tức, review): thuật ngữ hiếm
- Trộn tuyến tính α (0,6–0,8 cho dense) $>$ RRF — nhưng α phải tune trên golden set
- Âm tiết $\neq$ từ: "học sinh" tách khoảng trắng $\rightarrow$ "học" + "sinh". Tách từ (pyvi, underthesea) giúp BM25 bao nhiêu? **Chưa có số công bố — tự đo**

Filtered Search: Cái Bẫy Recall Ít Ai Nói Đến

VINUNIVERSITY

Post-filter

ANN trước → lọc sau.

- Xin top-100, filter khớp 2% ⇒ còn **2 kết quả**
- Recall sập **không báo lỗi**
- Càng lọc chặt càng tệ

Pre-filter (brute force)

Lọc trước → quét toàn bộ subset.

- Kết quả **luôn đúng**
- Nhưng **mất index**
- Latency tăng theo kích thước subset

Filtered-ANN (đúng)

Index tự biết filter.

- Qdrant: filterable HNSW payload index
- Milvus: partition key
- **pgvector 0.8:** iterative scan — tự nới rộng đến khi đủ *k* hàng

Lưu ý: "Lọc sớm cho nhanh" là bản năng *sai*. Filter phá tính liên thông của đồ thị HNSW ⇒ đi lạc, trả về ít và kém. **Bài test bắt buộc:** chạy golden set với filter *chọn lọc mạnh* (khớp ~1% corpus) — đây là lúc hệ thống gãy, không phải lúc query trống.

Late Interaction: ColBERT → ColPali (RAG cho PDF)

VINUNIVERSITY

Single-vector vs Late interaction

Single-vector: cả chunk nén thành *một* vector — chi tiết token bị bình quân hoá.

Late interaction: giữ vector *từng* token/-patch, chấm điểm bằng **MaxSim**: score $= \sum_{t \in q} \max_{d \in D} \cos(t, d)$

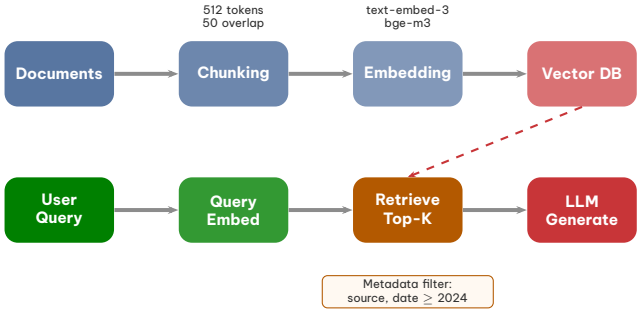
Mỗi token truy vấn tự tìm mảnh khớp nhất trong tài liệu ⇒ bắt được chi tiết mà single-vector bỏ sót.

ColPali: RAG không cần OCR

- Encode **ảnh trang PDF** trực tiếp (SigLIP-So400m)
- Lưới $32 \times 32 = 1024$ **patch/trang**, mỗi patch 128d
- Bảng biểu, biểu đồ, form scan: **hết là bài toán tiền xử lý**
- Họ model: ColQwen2.5, ColSmolVLM, ColInternVL

Lưu ý: **Chất lượng mua bằng dung lượng.** 1 triệu trang × 1024 patch × 128d × 4B ≈ **524 GB** (còn ∼30–60 GB sau quantization). Vector DB một-vector-một-document **không lưu nổi** — cần hỗ trợ multi-vector + MaxSim (Qdrant, Milvus).

RAG Pipeline: End-to-End Flow



Chunking Strategies: Quyết Định 80% Chất Lượng RAG

VINUNIVERSITY

4 Strategies

- **Fixed-size:** 512 tokens, baseline, ngắt giữa câu
- **Recursive:** LangChain RecursiveCharacterTextSplitter — tôn trọng câu/đoạn
- **Semantic:** similarity-based, nghỉ tại topic shift — chậm + tốt nhất
- **Hierarchical:** parent (page) + child (paragraph), retrieve child, return parent

Production Tuning

- **Chunk size:** 200–500 tokens cho RAG, 1K+ cho long-context
- **Overlap:** 10–20% (~50–100 tokens)
- **Tooling:** LangChain, LlamaIndex, Unstructured.io
- **Vietnamese:** tokenize ở sentence level, không byte-split

Lưu ý: **Đo bằng golden set:** Recall@k cải thiện đáng kể khi switch fixed → semantic trên benchmarks.
Bad chunking = embedding tốt cũng vô dụng.

RAG: Production Best Practices

VINUNIVERSITY

Retrieval Hyperparameters

- **Top-K:** 5–10 cho RAG, 20–50 trước reranker
- **ef_search** (HNSW): 200 default, tăng cho recall
- **MMR diversity:** λ=0.5 giảm chunks trùng
- **Metadata filter:** *không* phải cứ "lọc sớm" — xem slide Filtered Search

Production Gotchas

- Embedding model consistency: train = inference
- Query rewriting: HyDE, multi-query expansion
- Embedding versioning + zero-downtime re-index (Production Patterns)
- Monitor embedding drift hàng tuần

Lưu ý: Training và inference **phải** dùng cùng embedding model version. Đổi model = re-index toàn bộ.

2-Stage Retrieval: Vector Search + Reranking

VINUNIVERSITY

Stage 1: Recall (nhanh)

- ANN search trả về top-100 candidates
- Bi-encoder: query & doc embed **riêng**
- Latency thấp (<20 ms), recall cao

Stage 2: Precision (chính xác)

- Cross-encoder: query + doc **cùng lúc**
- Score relevance từng cặp (query, doc)
- Chọn top-10 chất lượng cao nhất

Rerankers 2026 — chọn theo latency budg

- **Self-host + GPU:** `bge-reranker-v2-m3`, Jina v3, Qwen3-Reranker — ∼50–200 ms
- **Managed API:** Cohere Rerank v4, zerank — ∼600 ms, không cần GPU

⇒ Rerank **cộng thẳng vào P99**

Embedding Model Hosting

- **Self-host:** sentence-transformers trên GPU — kiểm soát, tiết kiệm ở scale lớn
- **API:** OpenAI `text-embedding-3-small`, Cohere Embed 5 + Rerank v4 — đơn giản, triển khai nhanh

“Cửa sổ 1M token rồi, còn cần RAG không?”

Context Rot (Chroma Research, 07/2025)

18 model frontier, mở rộng từ Needle-in-a-Haystack:

- Chất lượng **giảm khi input dài ra** — kể cả task đơn giản, và giảm **không đều** giữa các model
- Model cửa sổ 1M: hiệu ứng thường thấy rõ quanh **300–400K token**
- Multi-hop / tổng hợp: gãy **sớm hơn nhiều**
- Phụ thuộc độ giống câu hỏi, nhiễu, cấu trúc — *đúng những biến mà retrieval kiểm soát*

Kinh tế học: một lần vs mỗi lần

Nhồi cả corpus: trả tiền cho *toàn bộ* corpus ở **mỗi** request.

Retrieval: trả tiền index **một lần**, mỗi query chỉ trả cho phần đã chọn.

Chênh lệch nhân theo QPS — đây là lý do RAG không biến mất.

Mặc định 2026

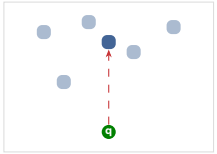
Retrieve **50K–200K token** liên quan, rồi mới để model long-context suy luận trên đó — **lai, không phải chọn một**.

Cửa sổ lớn là **sức chứa**, không phải **đảm bảo**.

GraphRAG: Khi Quan Hệ Quan Trọng Hơn Đoạn Văn

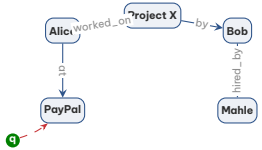
VINUNIVERSITY

Vector RAG (chunks)



Top-K nearest chunks
(cosine similarity)

GraphRAG (entity-relation)



Multi-hop: PayPal → Alice
→ Project X → Bob → Mahle

Câu hỏi mà vector RAG kém

"Ai ở PayPal từng cộng tác với người được Mahle thuê?" — Vector trả về chunks về **PayPal** HOẶC **Mahle** riêng lẻ; không thể **cross-document multi-hop**. KG traverse 3-hop trong μs.

Use Cases 2026

- **P3C diabetes copilot** (Memgraph): patient journey + drug interactions
- **Alzheimer research**: 1.6M edges nối genes-drugs-trials
- **M&A intel** (GlassDollar/Siemens, Mahle): entity search across millions of companies

F

Vector RAG = "đoạn văn liên quan". GraphRAG = "mối quan hệ kết nối nhiều entity" —

GraphRAG Construction: Document → Knowledge Graph



Tooling Landscape 2026

- **MS GraphRAG**: community summaries, chất lượng cao, index đắt nhất
- **LazyGraphRAG**: hoãn extraction sang query-time — index rẻ ngang vector RAG
- **LightRAG** / Fast GraphRAG: index nhẹ
- **Neo4j** (Cypher) · **Memgraph** (in-memory, sub-ms traversal)

Vector + Graph: Layered (2026 default)

1. **Start:** vector RAG cho conversational grounding
2. **Add KG:** domain nhiều thực thể (legal, medical, M&A)
3. **Hybrid:** vector retrieve → KG expand → LLM
4. **Cost:** full extraction 5–20×; Lazy ≈ vector RAG

Lưu ý: **Cost 2026 đã đảo chiều:** LazyGraphRAG hoãn phần đắt sang query-time ⇒ index **bằng vector RAG** ($0{,}1\%$ của full GraphRAG). "Graph quá đắt để thử" **không còn là lý do hợp lệ** — chọn theo *hình dạng câu hỏi*, không theo ngân sách index.

Agentic Retrieval: Truy Xuất Trở Thành Một Công Cụ

VINUNIVERSITY

Classic RAG vs Agentic retrieval

Classic RAG — một lượt: embed query → top-K → sinh câu trả lời. Dùng cho *tra cứu đơn giản*.

Agentic retrieval — agent *quyết định* truy xuất thế nào: phân tích độ phức tạp, tách câu hỏi nhiều phần, chọn nguồn, truy xuất nhiều vòng.

Cái giá: thêm LLM call ⇒ thêm latency + token. *Không* phải mặc định cho mọi query.

Vòng lặp agentic

1. **Hiểu query** — đơn giản hay nhiều bước?
2. **Lập kế hoạch** — cần nguồn nào, thứ tự nào
3. **Truy xuất** — vector / BM25 / KG / SQL / API
4. **Phản tỉnh** — đã đủ bằng chứng chưa?
5. **Lặp lại** nếu chưa đủ, rồi mới sinh câu trả lời

Hạ tầng đang mọc giao diện cho agent — Weaviate (1.37+, *Preview*) nhúng sẵn **MCP server** tại `/v1/mcp`, cùng cổng với REST API — agent truy vấn *và ghi* thẳng vào vector DB, không cần lớp tích hợp riêng. Vector DB không còn chỉ là thư viện phía sau, mà là **một công cụ agent gọi được** (xem Ngày 9: MCP).

Agent Memory & Semantic Cache: Hai Nửa Của Bài Hôm Nay

VINUNIVERSITY

Agent memory = chính hai kho của hôm nay

Online store trả lời "ta biết gì về user *này*" (<10 ms). **Vector store** trả lời "đã từng nói gì liên quan".

Ba kiến trúc 2026:

- **Mem0**: vector-first, trích "sự kiện" từ hội thoại
- **Zep**: knowledge graph *theo thời gian* (Graphiti)
- **Letta** (ex-MemGPT): 3 tầng core / recall / archival

Benchmark: LoCoMo, LongMemEval, BEAM.

Semantic cache — đòn bẩy chi phí rẻ nhấ

Query mới *gần* query cũ (cosine > ngưỡng) ⇒ trả lại câu trả lời đã lưu.

AWS đo trên 63.796 query thật (ngưỡng 0,75):

- Chi phí inference **giảm tới 86%**
- Latency **cải thiện 88%** trên cache hit
- Độ chính xác giữ ~91%

Lưu ý: **Cache sai nguy hiểm hơn không cache.** 4 kiểu gãy: query phụ thuộc ngữ cảnh (giống nhau nhưng cần đáp án khác) · query nhạy thời gian (trả lời cũ) · **đổi embedding model ⇒ vô hiệu toàn bộ cache** (đúng cái bẫy re-index) · cache nhầm một câu bịa rồi phục vụ mãi. Ngưỡng similarity là tham số phải *đo*, không

Retrieval Như Một Tool: Thứ Agent Thực Sự Nhìn Thấy

```
SEARCH_TOOL = {
  "name": "search_docs",
  # this description IS the retrieval prompt:
  # the agent decides *from this text* when to call
  "description": (
    "Search internal product docs. Use for questions "
    "about pricing, limits, API behaviour. Returns "
    "ranked chunks with source URLs."
  ),
  "input_schema": {
    "type": "object",
    "properties": {
      "query":   {"type": "string"},
      "product": {"type": "string",
                  "enum": ["core", "billing", "api"]},
      "top_k":   {"type": "integer", "default": 8},
    },
    "required": ["query"],
  },
}
```

Bốn điều quyết định chất lượng

- **`description` là prompt truy xuất** — agent chỉ có nó để quyết định gọi hay không. Mơ hồ ⇒ gọi sai lúc.
- **Filter là `enum`**, không phải string tự do ⇒ agent không bịa giá trị
- **Trả về citation + score**, không chỉ text — để LLM (và bạn) truy nguồn
- **`top_k` có trần** — nếu không, agent tự làm loãng context

Ghép Ngữ Cảnh: Nơi Feature Store Gặp Vector Store

```python
def build_context(user_id: str, question: str) -> str:
    # 1) WHO is this user?  online store, <10 ms
    f = store.get_online_features(
        features=["user_profile:topic_affinity",
                  "user_profile:preferred_language"],
        entity_rows=[{"user_id": user_id}],
    ).to_dict()

    # 2) WHAT is relevant?  vector search + filter
    hits = vdb.search(
        embed(question), top_k=8,
        filter={"lang": f["preferred_language"][0]},
    )

    # 3) personalise, THEN ground
    return PROMPT.format(
        affinity=f["topic_affinity"][0],
        docs="\n".join(h.text for h in hits),
    )
```

Hai câu hỏi khác nhau

Feature store trả lời *"user này là ai"* → cá nhân hoá.

Vector store trả lời *"cái gì liên quan"* → grounding.

Cá nhân hoá mà không grounding = **bịa có duyên**. Grounding mà không cá nhân hoá = **đúng nhưng vô hồn**.

Ngân sách

Feature lookup <10 ms — gần như *miễn phí* so với một LLM call. Đừng tiếc nó.

Semantic Cache: Hiện Thực Trong 12 Dòng

```python
def cached_answer(user_id, q, threshold=0.75):
    qv = embed(q)
    # namespace per tenant: NEVER share across tenants
    hit = cache.search(
        qv, top_k=1,
        filter={"tenant": tenant_of(user_id)},
    )
    if hit and hit[0].score >= threshold:
        return hit[0].payload["answer"]     # HIT

    ans = llm(build_context(user_id, q))    # MISS
    cache.upsert(
        qv, {"answer": ans, "q": q},
        ttl=3600,                           # must expire
    )
    return ans
```

Ba tham số, ba loại lỗi

- **threshold** quá thấp ⇒ trả nhầm câu trả lời của câu hỏi khác. *Phải đo, đừng đoán* (AWS dùng 0,75).
- **ttl** thiếu ⇒ câu trả lời cũ sống mãi
- **namespace** thiếu ⇒ user A nhận câu trả lời chứa dữ liệu user B — đây là **lỗ hổng bảo mật**, không phải bug cache

Lưu ý: Đổi embedding model ⇒ **xoá sạch cache**. Vector cũ và mới không cùng không gian.

Debug Agent + Retrieval: Triệu Chứng → Cách Sửa

Triệu chứng	Nguyên nhân thường gặp	Cách sửa
Agent gọi `search` 5–6 lần rồi bỏ cuộc	`description` mơ hồ, hoặc filter quá chặt trả về rỗng	Viết lại `description`; **log filter thực tế** agent sinh ra
Trả lời chung chung dù tài liệu đúng *có* trong top-K	`top_k` quá lớn → context loãng (context rot)	Giảm `top_k`, thêm reranker
Chạy đúng lúc demo, sai trên production	Query thật khác golden set	Log query thật, refresh golden set hàng tuần
Câu trả lời cũ dai dẳng	Semantic cache không TTL / không invalidate	TTL + xoá cache khi tài liệu nguồn đổi
Agent làm theo "lệnh" nằm trong tài liệu	Prompt injection qua retrieved doc	Tách data khỏi instruction; retrieved text **không được** điều khiển tool

Feature Engineering: 6 Họ Feature Bạn Sẽ Viết Đi Viết Lại

VINUNIVERSITY

Bốn họ "kinh điển"

1. **Aggregation theo cửa sổ** — `count/sum/avg` 5 phút, 1 giờ, 7 ngày. *Xương sống của fraud & recsys.*
2. **Tỷ lệ & chuẩn hoá** — `amount / avg_7d` của *chính user đó*. Bắt bất thường **tương đối**.
3. **Lag & delta** — giá trị kỳ trước, độ thay đổi. Cho xu hướng.
4. **Recency** — `now − last_event`. Feature rẻ nhất mà mạnh bất ngờ.

Hai họ "dễ sai"

5. **Mã hoá categorical** — one-hot (ít giá trị), frequency/target encoding (nhiều giá trị). **Target encoding phải fit trong fold**, nếu không là leakage.
6. **Embedding làm feature** — vector user/item đi thẳng vào model. Cầu nối sang nửa đầu bài hôm nay.

Mẹo đặt tên

`<entity>_<phép tính>_<cửa sổ>` — ví dụ `user_txn_count_7d`. Tên tự mô tả cửa sổ thì PIT join và debug đỡ đau.

Từ Ý Tưởng Đến FeatureView: Code Thật (Lab 19)

```python
from datetime import timedelta
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Int64

user = Entity(name="user", join_keys=["user_id"])

src = FileSource(
    path="data/query_velocity.parquet",
    timestamp_field="event_timestamp",  # PIT join key
)

query_velocity_features = FeatureView(
    name="query_velocity_features",
    entities=[user],
    ttl=timedelta(hours=1),             # PIT window; online: per store
    schema=[Field(name="queries_last_hour", dtype=Int64)],
    source=src,
    online=True,                        # -> online store
)
```

Bốn quyết định trong 20 dòng

- `entities` — khoá tra cứu lúc serving
- `timestamp_field` — **không có nó thì không có PIT join**, và bạn sẽ leak
- `ttl` — "cũ bao lâu thì vô nghĩa"; lúc serve **chỉ Redis/ScyllaDB chặn** giá trị cũ, SQLite thì không
- `online=True` — tốn tiền, chỉ bật khi serving cần

`feast apply` → `feast materialize-incremental`

On-Demand Feature: Tính Tại Thời Điểm Request

```python
from feast import Field, RequestSource
from feast.types import Float64
# feast 0.65: importing this from `feast` gives the MODULE
from feast.on_demand_feature_view import on_demand_feature_view

# amount exists only at request time
txn = RequestSource(
    name="txn",
    schema=[Field(name="amount", dtype=Float64)],
)

@on_demand_feature_view(
    sources=[user_spend_stats, txn],  # stored + request
    schema=[Field(name="amount_vs_avg", dtype=Float64)],
    mode="python",
)
def amount_vs_avg(inputs):
    pairs = zip(inputs["amount"], inputs["avg_amount_7d"])
    return {"amount_vs_avg":
            [a / m if m else 0.0 for a, m in pairs]}
```

Vì sao cần on-demand?

Số tiền giao dịch **chưa tồn tại** lúc materialize — không thể pre-compute. Nhưng `amount/avg_7d` lại là feature mạnh nhất của fraud.

On-demand ghép *feature đã lưu* với *dữ liệu request* và áp **cùng một công thức** cho cả training lẫn serving.

Lưu ý:
`write_to_online_store=True` ⇒ tính lúc *ghi*; mặc định `False` ⇒ lúc *đọc*.

Tabular Foundation Model: Còn Cần Feature Engineering?

TabPFN, TabICL: model huấn luyện sẵn trên hàng triệu bảng tổng hợp, dự đoán bằng *in-context learning* — không train lại cho từng bộ dữ liệu.

Đo trên event log Lab 19 (9.019 dòng, chia theo thời gian 80/20, nhãn `clicked`, test AUC):

Model	cột thô	+ aggregate	fit+predict
HistGradientBoosting	0,537	0,677	0,6 s
TabICL (CPU)	0,557	**0,693**	∼35 s

Đổi model: **+0,02**. Thêm feature: **+0,14**.

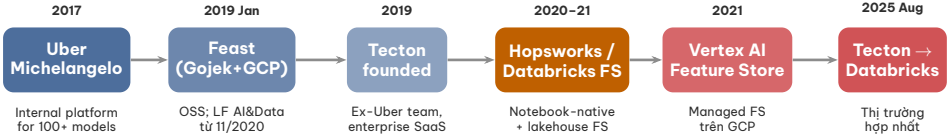
Vì sao: nhãn phụ thuộc *hành vi gần đây của từng user*; cột thô không chứa thông tin đó — **model không học được thứ bạn không đưa vào**.

Trước khi đưa vào production

- **Latency:** 151–1.275 ms trên GPU, trong khi fraud cần <2 ms $\Rightarrow$ chưng cất sang XGBoost: 1,9 ms CPU, giữ 96,5% AUC (arXiv 2605.18654)
- **Kích thước:** TabPFN-2.5 thiết kế cho $\leq$50K dòng — bảng event 100M dòng phải *nén bằng aggregate* trước
- **Giấy phép:** weights TabPFN-2.5 **cấm dùng thương mại**; TabICL weights BSD-3

Feature store vẫn giữ phần FM không làm: PIT đúng, aggregate theo entity, lookup $<$10 ms.

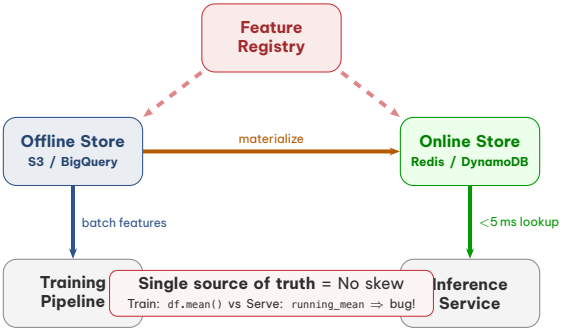
Feature Stores: Uber Michelangelo → Hợp Nhất 2025



Why feature stores emerged

Uber có 100+ ML models reuse cùng features (rider price, driver ETA) ⇒ centralize để tránh skew + duplicate compute. **LLM-era twist:** feature stores giờ host cả *embedding feature views* (user/item vectors) bên cạnh tabular features — một hệ thống cho cả ML cổ điển và RAG personalization.

Feature Store Architecture



Feast: Define & Serve Features

Feature Definition

- `FeatureView(name="user_features")`
- Entities: `["user_id"]`
- TTL: `timedelta(days=30)`
- Source: Delta/Iceberg tables (N18)

Materialize & Freshness

- `feast materialize-incremental`
- Batch features: daily update là đủ
- Streaming features: cần sub-second
- **Feast 0.65** (07/2026, LF AI&Data — release *hàng tháng*, phải pin version): Push API, on-demand transformations (Beta), streaming transformations (Alpha)

Online Lookup

- `store.get_online_features()`
- Latency: <5 ms per request
- Batch: 1000 entity rows at once

Feast Alternatives

- **Tecton:** real-time managed — nay thuộc Databricks (2025)
- **Databricks:** Feature Engineering in Unity Catalog (Workspace FS đã legacy)
- **Vertex AI FS:** GCP managed service
- **Hopsworks:** regulated industries, on-prem

Online vs Offline Store + Point-in-Time Join

Offline Store (training)

- Parquet trên S3 / Delta / Iceberg
- Snowflake / BigQuery / Redshift
- **Workload:** historical batch, 100GB–TB
- **Latency:** seconds–minutes (OK for training)
- Lưu *full history* của mọi feature value

Online Store (serving)

- Redis / DynamoDB / Cassandra / Aerospike
- **Workload:** per-entity KV lookup
- **Latency:** $<$10 ms P99
- Lưu *current value only* (no history)
- Feast `materialize()` nạp offline $\rightarrow$ online

Point-in-Time (PIT) Join – Khi build training set: lấy feature *as-of timestamp* của mỗi event – không dùng giá trị tương lai. Feast `get_historical_features(entity_df)` thực hiện PIT join trên offline store. **Sai lầm:** dùng LATEST value $\Rightarrow$ data leakage $\Rightarrow$ prod accuracy thấp hơn training 20–30%.

Độ Tươi Của Feature ≠ TTL

`ttl` làm hai việc khác nhau

- **Offline (train):** cửa sổ nhìn lùi của PIT join — feature cũ hơn `ttl` so với event ⇒ null
- **Online (serve):** chặn giá trị cũ khi đọc — **tuỳ store**. Bảng chức năng của Feast: *chỉ Redis và ScyllaDB* làm việc này; SQLite (Lab 19) thì không

Đo được (Feast 0.66 + SQLite) —

Ghi `queries_last_hour = 7` với timestamp **50 giờ trước**, view có `ttl=1h` → `get_online_features` **vẫn trả về 7**.

Freshness SLO: tự đo tuổi feature

age = *now* − *event_ts*, lấy *event_ts* bằng `to_dict(include_event_timestamps=True)` → cột `<feature>__ts`.

Ví dụ: SLO = 1 giờ; 4 request có tuổi 10 phút, 40 phút, 2 giờ, 50 giờ ⇒ **2/4 = 50%** vi phạm.

Guard: *age* > SLO ⇒ dùng giá trị mặc định hoặc model dự phòng *và* đếm metric.

Lưu ý: Job streaming chết ⇒ feature **đóng băng** ở giá trị cuối, model vẫn trả lời tự tin. Đây là silent failure (Ngày 10), và `ttl=1h` trong code không bảo vệ bạn.

Feature Stores 2026: Ba Kiến Trúc, Không Phải Ba Vendor

VINUNIVERSITY

Framework mã nguồn mở

- **Feast 0.66** (08/2026), LF AI&Data từ 2020
- Bạn tự chọn store: SQLite, Redis, Postgres, Milvus…
- Cùng registry cho feature bảng *và* vector
- **Hợp khi:** multi-cloud, tự vận hành được

Gắn với lakehouse

- **Databricks** FE + online store trên Lakebase — Tecton nhập vào (2025)
- **Snowflake** Online Feature Store (preview 07/2026, Postgres)
- Feature = bảng trong catalog (Ngày 18)
- **Hợp khi:** data đã nằm ở lakehouse đó

Nền tảng managed riêng

- **Hopsworks 5.0** (06/2026): on-prem + cloud, governance
- **SageMaker** Feature Store
- **Vertex AI**: optimized online serving **dừng 17/02/2027** → Bigtable
- **Hợp khi:** ngành có quy định, cần on-prem

Lưu ý: **Thị trường đang hợp nhất:** Tecton (08/2025) và Fennel → Databricks, Featureform → Redis. Câu hỏi đúng không phải "vendor nào" mà là *feature của bạn sống ở đâu, ai giữ online store, store đó có thực thi TTL không*.

Training-Serving Skew: Lỗi Thầm Lặng

Skew là gì?

Feature tính **khác nhau** giữa training và serving dẫn đến model hoạt động kém khi deploy mà không có lỗi rõ ràng.

Ví dụ 1: Aggregation

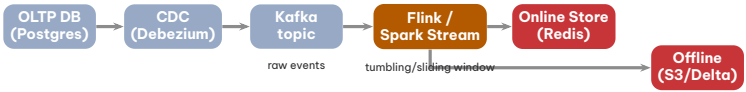
- **Training:** `pd.DataFrame.mean()` trên toàn bộ dữ liệu lịch sử
- **Serving:** `running_mean(last_N)` chỉ N bản ghi gần nhất
- ⇒ Giá trị khác nhau, model sai lệch

Ví dụ 2: Date Parsing

- **Training:** parse datetime UTC
- **Serving:** parse local timezone
- ⇒ Feature "giờ trong ngày" lệch 7h (VN)

Lưu ý: Feature Store giải quyết skew: **một định nghĩa duy nhất** dùng cho cả training lẫn inference — không code riêng.

Streaming Feature Pipelines: Sub-Second Freshness



Streaming patterns

CDC: mọi UPDATE row → Kafka event

Aggregate: txn velocity 5min / 1h window

Feast Push API: app push trực tiếp event → online store

Khi nào cần streaming

Có: fraud detection, dynamic pricing, real-time recsys

Không cần: churn prediction (daily batch OK), credit scoring

Feature Store trong ML Production: 3 Use Cases

VINUNIVERSITY

Fraud Detection

- **Latency:** <10 ms (per transaction)
- **Features:** velocity (txn/min), device fingerprint, amount-vs-7d-avg
- **Freshness:** streaming (Kafka → Redis online store)
- **Mẫu:** stream → online store in-memory (Redis/Aerospike) — store *phải* chặn được giá trị quá TTL

Recommendation Systems

- **Latency:** <50 ms (homepage personalization)
- **Features:** user embedding, last-N interactions, item popularity
- **Freshness:** hybrid (daily user-batch + stream last click)
- **Ví dụ thật:** DoorDash tự xây (Flink → Redis feature store), không mua platform

Dynamic Pricing

- **Latency:** <100 ms (price at search)
- **Features:** demand surge, supply, competitor price, time-of-day
- **Freshness:** sub-second (streaming demand signal)
- **Mẫu:** streaming aggregate + on-demand feature tính lúc request

Feature Store thắng ở **3 dimensions**: feature reuse + train/serve consistency + low-latency lookup — không tool nào khác giải quyết cả 3.

Hội Tụ: Feature Store *Chính Là* Vector Store

Feast 0.65: online store có thể là vector DB

- Online store nay gồm cả **Qdrant, Milvus, FAISS** — bên cạnh Redis, DynamoDB, Cassandra, Postgres…
- **Vector search** trong Feast: trạng thái *Alpha*, nằm dưới nhánh roadmap "NLP"
- On-demand transformations (Beta), streaming transformations (Alpha)

Nghĩa là gì với kiến trúc của bạn?

Một registry phục vụ đồng thời:

- feature dạng bảng (`txn_count_7d`)
- **embedding feature view** (vector user/item)

…cho *cùng* một model, qua *cùng* một lần online lookup — cùng định nghĩa, cùng PIT join, cùng chống skew.

D Hai nửa của bài hôm nay đang nhập làm một: RAG cần *ngữ cảnh cá nhân hoá* (feature), còn ML cổ điển ngày càng dùng *embedding* làm feature. Đừng dựng hai hệ thống song song nếu một cái đủ.

Embedding Là Một Feature: `vector_index` Trong Feast

```python
# 1) an embedding declared like any other field
document_embeddings = FeatureView(
    name="embedded_documents",
    entities=[item],
    schema=[
        Field(name="vector", dtype=Array(Float32),
              vector_index=True,
              vector_search_metric="COSINE"),
        Field(name="sentence_chunks", dtype=String),
    ],
    source=rag_documents_source,
    ttl=timedelta(hours=24),
)

# 2) retrieved by similarity, not by key
ctx = store.retrieve_online_documents_v2(
    features=["embedded_documents:vector",
              "embedded_documents:sentence_chunks"],
    query=query_embedding, top_k=3,
).to_df()
```

Điều gì vừa xảy ra?

Cùng `FeatureView` API, cùng registry, cùng TTL — nhưng tra cứu bằng **độ tương đồng** thay vì bằng **khoá**.

RAG corpus giờ là một *feature view*: có schema, lineage, versioning, PIT semantics.

```
online_store:
type: milvus
vector_enabled: true
```

Alpha — Milvus, SQLite, Qdrant, PGVector.

Vector Search: Production Optimization

Caching & Batching

- **Embedding cache:** Redis TTL 24h
- Hit rate 60–80%, giảm embedding cost
- **Batch embedding:** 1000 texts/request
- Throughput 50×, cost giảm 90%

Monitoring Search Quality

- Track relevance score distribution theo tuần
- Nếu avg similarity giảm: điều tra embedding drift, data quality, query distribution
- P99 latency + index size growth

Multi-tenancy

- Namespace isolation (Pinecone)
- Collection per tenant (Qdrant)
- Security + billing separation
- **Cảnh báo:** lọc bằng metadata = isolation *mềm*; 1 bug filter ⇒ rò dữ liệu chéo tenant (OWASP LLM09:2026 — xem slide Security)

Index Lifecycle Management

- Create V1 → ingest → serve
- Update: build V2 offline → validate quality
- Blue-green switch → retire V1
- Rollback ngay nếu quality giảm

Retrieval Evaluation & Observability

Offline Metrics (Golden Set)

- **Recall@k**: tỷ lệ relevant docs trong top-k
- **MRR**: vị trí trung bình của kết quả đúng đầu tiên
- **nDCG@k**: xếp hạng chất lượng có weight
- Build 200-query golden set — regression-test mỗi lần đổi embedding hoặc chunk strategy

RAG-specific: LLM-as-Judge

- **Context Relevance**: retrieved chunks có liên quan không?
- **Answer Relevance**: câu trả lời có trả lời query không?
- **Groundedness**: câu trả lời có dựa trên context không?

⇒ RAGAS: faithfulness, answer relevancy, context precision

Online Metrics (Production)

- P99 search latency + embedding latency
- Embedding cache hit rate (target >60%)
- Query distribution drift (alert nếu avg similarity giảm)
- Index size growth rate (trigger re-balance khi >120%)

Observability Tooling

- **Langfuse** (open-source, OTel): tracing + eval tích hợp
- **Phoenix/Arize**: OpenTelemetry-native, RAG tracing
- **LangSmith**: LangChain-native, prompt + retrieval debug
- Chọn 1 tool — instrument từ ngày đầu tiên

Golden Set: Chấm Retrieval Bằng Nhãn Có Mức Độ

Xây golden set trong 4 bước

1. Query **thật** từ log + query sinh tự động từ tài liệu
2. **Pooling:** gộp top-k của nhiều hệ để chấm, không chỉ hệ của bạn
3. Nhãn **0–3** (không liên quan → trả lời trọn vẹn), không chỉ đúng/sai
4. LLM-judge **chỉ sau khi** đo đồng thuận với người

Bằng chứng: TREC 2024 RAG

LLM-judge (UMBRELA) **xếp hạng các hệ** gần giống người chấm — nhưng không thay được từng nhãn, và **người chấm khắt khe hơn** LLM (arXiv 2411.08275).

Tính tay: nDCG@5 và κ

Kết quả trả về có nhãn $[0, 3, 2, 0, 1]$:

DCG $= \frac{3}{\log_2 3} + \frac{2}{\log_2 4} + \frac{1}{\log_2 6} = 1{,}893 + 1{,}000 + 0{,}387 = 3{,}280$

Lý tưởng $[3, 2, 1, 0, 0]$: $3 + 1{,}262 + 0{,}500 = 4{,}762$

$\Rightarrow$ **nDCG@5 = 0,689**. Nhãn nhị phân cho 0,712 — che mất việc tài liệu *tốt nhất* nằm ở hạng 2.

Judge khớp người **75%**, ngẫu nhiên đã khớp 50%: $\kappa = \frac{0{,}75 - 0{,}50}{1 - 0{,}50} =$ **0,50** — "khá", không phải "75% đúng".

Lưu ý: Query sinh từ *cùng template* với corpus $\Rightarrow$ hybrid "thắng" sẵn (Lab 19 cố ý như vậy).

Bảo Mật & Tuân Thủ: Vector Store Là Kho Dữ Liệu Cá Nhân

OWASP LLM09:2026 — Vector & Embedding

- **Embedding inversion:** tái dựng văn bản gốc từ vector — "*Embeddings only* is not a safe-harbor"
- **Rò chéo tenant:** "enforce tenant scoping *inside the index query*, not as a post-retrieval filter"
- **Poisoning & jamming:** tài liệu độc lái câu trả lời, hoặc ép agent từ chối
- **Mới 2026:** membership inference qua similarity score; đầu độc semantic cache

Lưu ý: **Quyền được xoá gặp index ANN.** Yêu cầu xoá phải lan tới *index vector* + *online store* + *semantic cache*. Thiết kế đường xoá **trước khi** ingest.

Luật Việt Nam đã có hiệu lực

PDPL — Luật 91/2025/QH15 (hiệu lực **01/01/2026**), Nghị định 356/2025: quyền chủ thể dữ liệu, **DPO bắt buộc**, đánh giá tác động, **báo cáo vi phạm trong 72 h** (24 h nếu bị tấn công hệ thống), chế tài *hình sự*. Áp dụng cả tổ chức nước ngoài xử lý dữ liệu tại VN.

Luật AI — 134/2025/QH15 (hiệu lực **01/03/2026**), tham chiếu EU AI Act. Chuyển tiếp đến 01/03/2027 (y tế, giáo dục, tài chính: 01/09/2027).

Chi Phí Một Hệ Retrieval: Cộng Đủ 5 Khoản

5 khoản chi

1. **Embedding** — một lần cho corpus + delta mỗi ngày
2. **Lưu index** — dims × bytes × số vector, *sau* quantization
3. **Query compute** — QPS × tầng lưu trữ (RAM/SSD/S3)
4. **Reranker** — mỗi query, cộng thẳng vào P99
5. **LLM generation** — thường là khoản *lớn nhất*

5 đòn bẩy (theo thứ tự hiệu quả)

1. **Semantic cache** — cắt cả khoản 4 và 5
2. **Quantization** — int8 $4\times$, binary $32\times$ trên khoản 2
3. **Chọn đúng tầng** — object storage cho corpus ít truy vấn
4. **Matryoshka** — cắt dims, cắt luôn RAM
5. **Top-K nhỏ hơn** — ít token vào LLM hơn

Lưu ý: Khoản không ai lập ngân sách: **re-index**. Đổi embedding model = trả lại *toàn bộ* khoản 1 + dựng index song song (blue-green) = tạm thời **gấp đôi** khoản 2. Hãy coi nó là chi phí định kỳ, không phải sự cố. (FinOps sâu hơn: Ngày 25.)

Enterprise Case Studies: ROI từ Vector + Feature Store

40%
GlassDollar (Qdrant) hạ tầng cost ↓

3×
GlassDollar user engagement

1.6M
Alzheimer KG edges (Memgraph)

2025
Tecton → Databricks (22/08/2025)

Vector wins

GlassDollar (Qdrant): NL search Siemens/Mahle, 40% cost ↓, 3× engagement.

Memgraph P3C: KG 1.6M edges, sub-ms multi-hop traversal cho Alzheimer + diabetes.

Feature Store wins

DoorDash: feature store tự xây trên Redis, feature real-time qua Flink — *build*, không *buy*.

Thị trường 2025–26: Tecton, Fennel → Databricks; Featureform → Redis. Học *kiến trúc*, đừng học tên vendor.

H Số ROI ở trên do **vendor tự công bố** — con số đáng tin nhất là con số bạn tự đo.

Vector & Feature Store: ML Era vs LLM Era

ML Era (2015–2022)

- **Vector store role:** candidate generation cho recsys; retrieval cho face / image search
- **Embeddings:** task-specific (FaceNet, two-tower recsys, doc2vec); train per-app
- **Index tooling:** FAISS / Annoy / NMSLIB embedded trong app process
- **Feature store:** tabular features (counts, ratios, lags) cho XGBoost / DNN tabular
- **Freshness:** daily / hourly batch là mặc định; streaming chỉ cho fraud

LLM Era (2023–2026)

- **Vector store role:** RAG retrieval, agent memory, semantic cache
- **Embeddings:** foundation models (OpenAI, BGE, Voyage); một model nhiều use case
- **Index tooling:** managed multi-tenant (Pinecone, Zilliz Cloud, Vertex Vector)
- **Feature store:** tabular *plus* embedding views; on-demand transforms cho prompt context
- **Freshness:** sub-second streaming là baseline cho fraud, pricing, real-time recsys

Wiki / CodeWiki / DocWiki: RAG Trên Chính Repo Của Bạn

VINUNIVERSITY

Wiki (repo → trang)

Sinh tài liệu *tự động* từ source code.

DeepWiki (Cognition): đổi `github.com` thành `deepwiki.com` là có wiki. Hơn **50.000 repo** đã index (Cognition tự công bố), kèm sơ đồ Mermaid và **MCP server** để agent truy vấn (Ngày 9).

CodeWiki (hỏi về code)

Hỏi bằng ngôn ngữ tự nhiên, trả lời kèm **trích dẫn file + dòng**.

"Hàm nào xử lý retry?" → đoạn code thật, không phải đoán.

Chính là 2-stage retrieval + citation của §4, áp lên code.

DocWiki (docs cho agent)

Tài liệu được viết để *máy* đọc, không chỉ người.

`llms.txt`, docs-as-code, markdown thay HTML.

Mintlify (tự công bố): gần **một nửa** lưu lượng vào trang tài liệu nay đến từ *agent*.

E Cả ba **không phải sản phẩm mới** — chúng là đúng những mảnh của hôm nay: chunk → embed → index → hybrid search → rerank → agentic retrieval → sinh câu trả lời có trích dẫn. Bạn đã đủ kiến thức để tự dựng một cái.

CodeWiki: Vì Sao Chunk Code Khác Chunk Văn Bản

VINUNIVERSITY

Chia theo ký tự = hỏng

Text splitter cắt theo số ký tự sẽ **cắt đôi một hàm**:

- Embedding của nửa hàm không mang nghĩa gì
- Mất `import`, mất signature, mất scope
- Retrieve về một mảnh *không chạy được*

Văn bản chịu được cắt giữa câu. Code thì không — nghĩa nằm ở **khối**, không ở dòng.

Chia theo AST = đúng

Cắt tại **ranh giới cú pháp** (hàm, class, method) bằng `tree-sitter`; mỗi chunk mang theo scope chain, imports, signature.

cAST (CMU, arXiv 2506.15655): chunk theo AST cho **+4,3 điểm Recall@5** trên RepoEval và **+2,67 Pass@1** trên SWE-bench.

Nối lại với §5 GraphRAG — Code *vốn dĩ* là một đồ thị: hàm gọi hàm, module import module, class kế thừa class. Câu hỏi "đổi hàm này thì hỏng chỗ nào?" là câu hỏi **multi-hop** — đúng loại mà vector RAG kém và graph traversal mạnh. Nên CodeWiki tốt = vector (tìm đoạn liên quan) + call graph (lần theo phụ thuộc).

DocWiki: Tài Liệu Giờ Được Đọc Bởi Agent

`llms.txt` — sitemap cho LLM

Một file markdown ở `/llms.txt` chỉ cho agent biết nội dung nằm ở đâu, thay vì bắt nó crawl HTML.

- Phân cấp: file gốc trỏ tới index từng mục ⇒ agent chỉ lấy phần cần, **tốn ít token hơn**
- Mặc định trên Mintlify, Fern, GitBook, Vercel, Supabase
- **Chưa phải chuẩn** IETF/W3C — vẫn là đề xuất cộng đồng

Viết docs khác đi thế nào?

- **Markdown** thay HTML nặng
- Mỗi trang **tự đứng được** — agent hiếm khi đọc trang trước đó
- Ví dụ code **chạy được**, có import đầy đủ
- Tiêu đề mô tả *nhiệm vụ*, không phải marketing

Lưu ý: Đây là **chunking chiến lược** ở tầng tổ chức: bạn đang quyết định trước hệ thống retrieval của người khác sẽ cắt tài liệu của bạn thế nào. Trang viết rời rạc, phụ thuộc ngữ cảnh trang khác ⇒ chunk vô nghĩa ⇒ agent trả lời sai *về sản phẩm của bạn*.

Demo: Lab 19 — Hybrid Search + Feast

1. **Index 1.000 tài liệu** tiếng Việt vào Qdrant (in-memory) với `bge-small-en` (384d, CPU) — đổi model bằng `EMBEDDING_BACKEND`
2. **So sánh:** keyword vs semantic vs hybrid trên golden set 50 câu — và xem semantic yếu ở câu paraphrase tiếng Việt (model tiếng Anh)
3. **Đo:** P@10 và P99 latency qua FastAPI `/search`
4. **Feature Store:** Feast materialize → online lookup; ghi một giá trị 50 giờ tuổi rồi đọc lại với `ttl=1h`

Glossary: Vector & Feature Store Terminology

VINUNIVERSITY

Vector Store

- **Embedding:** dense vector biểu diễn nghĩa
- **Chunk:** đoạn text trước khi embed
- **Index:** cấu trúc ANN (HNSW, IVF, DiskANN)
- **Recall@k:** % relevant docs trong top-k
- **Quantization:** fp32 → int8/binary
- **Hybrid search:** BM25 + Vector → RRF merge
- **Reranker:** cross-encoder rerank top-N
- **Re-index:** rebuild khi đổi embedding model
- **Filtered-ANN:** index biết filter, tránh sập recall
- **Late interaction:** giữ vector từng token, chấm MaxSim
- **Embedding inversion:** dựng lại text gốc từ vector

Feature Store

- **Entity:** primary key (user_id, item_id)
- **Feature view:** schema + entity + source
- **Online / Offline store:** KV <10ms / full history
- **Materialize:** batch load offline → online
- **PIT join:** as-of timestamp, no leakage
- **Train-serve skew:** feature mismatch bug
- **Push API:** stream event → online store
- **Feature service:** bundled features cho 1 model
- **Registry:** Feast metadata catalog
- **Embedding feature view:** vector làm feature
- **Agent memory:** online store + vector store cho agent
- **Semantic cache:** tái dùng câu trả lời cho query gần

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. Hybrid Search (BM25 + Vector, fusion tune trên golden set) là mặc định production 2026 — nhưng **tầng lưu trữ** (RAM/SSD/object storage) là quyết định chi phí lớn nhất *của hạ tầng retrieval* (LLM generation vẫn là khoản lớn nhất cả hệ).

2. GraphRAG khi câu trả lời là **relationship** (multi-hop, cross-document), không phải đoạn text. Kiến trúc layered: vector → KG.

3. Feature Store thắng cùng lúc **3 dimensions**: feature reuse, train/serve consistency, low-latency online lookup — không tool nào khác giải quyết cả 3.

4. **Đo, đừng tin cấu hình:** chất lượng retrieval đo bằng golden set có nhãn mức độ; độ tươi feature đo bằng *tuổi* lúc serve — `ttl` trong code không phải bảo đảm.

Tiếp theo & Bài tập

Ngày 20: Model Serving & Inference Optimization

"Model accuracy 95% nhưng latency 3 giây — user đợi không nổi. Quiz + Milestone 1."

- Hoàn thành Lab 19: Vector & Feature Store
- Đọc trước: vLLM docs — PagedAttention paper
- Ôn tập Chương 4 cho Quiz N20

Hỏi & Đáp

Câu hỏi về Vector DB, Hybrid Search, GraphRAG, hay Feature Store?

VINUNIVERSITY

Cảm ơn!

AICB-P2T2 · Ngày 19

Vector Store & Feature Store

lms.vinuni.edu.vn · Slide & template trên LMS